

PHY Knowledge Distillation: Scalable Stochastic Abstraction for Wireless Systems

Sian Jin

Abstract—High-fidelity physical-layer (PHY) simulation is essential for accurate system-level evaluation of modern wireless systems but is computationally prohibitive under long time horizons and adaptive configurations. Existing PHY abstraction techniques reduce complexity but either neglect temporal correlation or rely on configuration-specific calibration and parameter tables that do not scale to realistic adaptive systems. In this work, we introduce *PHY Knowledge Distillation*, a new framework that replaces per-configuration calibration with learning-based parameter generation. A neural student model learns to infer the parameters of an interpretable stochastic effective-SINR process directly from the PHY configuration, treating the high-fidelity simulator as a stochastic teacher. The resulting model generates time-correlated effective SINR samples without channel realization generation, post-equalization SINR computation, or parameter lookup tables, while supporting smooth configuration adaptation and efficient time skipping. By shifting from calibration-and-storage to inference-and-generation, PHY knowledge distillation provides a scalable, interpretable, and computationally efficient foundation for next-generation system-level wireless simulation.

Index Terms—PHY knowledge distillation, PHY abstraction, stochastic process modeling, model-based generative learning, scalable wireless system simulation

I. INTRODUCTION

High-fidelity physical-layer (PHY) simulation plays a critical role in the design and evaluation of modern wireless systems. By explicitly modeling channel realizations, multi-antenna processing, interference, and receiver algorithms, full PHY simulators provide accurate packet error behavior across a wide range of operating conditions. However, this accuracy comes at a significant computational cost, making large-scale system-level simulations with long time horizons, adaptive protocols, and dense configuration spaces prohibitively expensive.

To mitigate this challenge, PHY abstraction techniques have been widely adopted. Classical link-to-system mapping methods, such as effective SINR mapping (e.g., EESM), decouple detailed PHY processing from system-level simulation by summarizing frequency-selective fading and MIMO effects into a scalar effective SINR, which is then mapped to packet error probability using AWGN lookup tables. While these abstractions substantially reduce simulation complexity, they typically treat effective SINR samples as temporally independent and therefore fail to capture temporal correlation and bursty packet error behavior—key characteristics that strongly influence higher-layer performance.

In prior work, we took an important step toward addressing this limitation by demonstrating that the logarithm of effective SINR can be accurately modeled as a low-order autoregressive (AR) stochastic process under fixed PHY configurations.

This EESM-log-AR formulation explicitly captures temporal correlation and reproduces burst error statistics observed in full PHY simulations, while achieving orders-of-magnitude runtime reduction. As a result, it enables efficient system-level evaluation that preserves essential temporal dynamics.

Despite these advantages, EESM-log-AR suffers from fundamental limitations that restrict its applicability in realistic, adaptive systems. The parameters of the AR process depend on a large set of configuration variables—including channel model, antenna configuration, bandwidth, modulation and coding scheme (MCS), number of spatial streams, and average SNR—and must be estimated independently for each operating point. Consequently, practical deployment requires extensive offline, often human-driven calibration campaigns and the maintenance of large configuration-indexed parameter tables. Moreover, table-based parameterization does not naturally support online configuration changes such as rate adaptation or spatial stream switching, leading to ad hoc handling and limited generalization. These challenges reveal a deeper issue: while EESM-log-AR captures the structure of effective SINR dynamics, it relies on per-configuration parameter calibration rather than learning how those dynamics vary with system conditions.

In this paper, we propose a fundamentally different approach by formulating effective SINR modeling as a *stochastic process knowledge distillation* problem. Rather than calibrating abstraction parameters for each PHY configuration, we treat the high-fidelity PHY simulator as a teacher that implicitly defines the stochastic behavior of effective SINR. A neural student model learns a shared, configuration-conditioned stochastic process whose parameters are inferred directly from the PHY configuration and link state. The autoregressive process structure itself is retained as a fixed, interpretable simulator, while the neural network amortizes parameter estimation across the configuration space. As a result, the proposed framework eliminates configuration-specific parameter tables, removes the need for offline calibration campaigns, and enables fast, on-the-fly adaptation to new or previously unseen configurations, with inference cost independent of MIMO dimension and bandwidth.

Specifically, we model the logarithm of effective SINR as a conditional, time-varying AR process whose parameters depend smoothly on the PHY configuration. Stability of the AR dynamics is guaranteed through partial autocorrelation parameterization, ensuring valid behavior across all operating conditions. To capture non-Gaussian residual effects arising from fast fading, interference, and decoding randomness, we model the innovation process using a conditional normalizing flow. Furthermore, we adopt a compositional conditioning

strategy that separates static channel and antenna characteristics from dynamic link-adaptation variables, enabling robust generalization across modulation, coding, and spatial stream configurations.

Beyond efficiency and accuracy, the proposed formulation offers important advantages in interpretability and simulator integration. Unlike generic sequence models such as recurrent neural networks, diffusion models, or autoregressive transformers, the log-domain AR process admits a clear physical interpretation: AR coefficients directly characterize temporal correlation induced by channel dynamics and interference persistence, while the innovation process captures fast-varying residual uncertainty. This structure enables inspection, validation, and debugging—properties that are essential in simulator-driven research and system design.

Equally important, the AR-based formulation naturally supports efficient multi-step prediction and time skipping. Because state evolution follows a linear recurrence in the log domain, the process can be analytically propagated over multiple packet intervals without simulating every intermediate step. This capability is critical for large-scale system simulations, where events of interest such as scheduling decisions, rate adaptation updates, or retransmissions may occur sparsely in time. In contrast, generic black-box generative models typically require step-by-step sampling or iterative refinement, making them ill-suited for fast-forward simulation.

By retaining an explicit stochastic process structure while learning how its parameters adapt to system conditions, the proposed PHY knowledge distillation framework achieves a favorable balance between physical interpretability, computational efficiency, temporal fidelity, configuration adaptivity, and calibration scalability. This makes it a practical and extensible foundation for next-generation system-level wireless simulation.

II. PROBLEM STATEMENT

A. Effective SINR Generation as a Stochastic Process

Consider a wireless link operating under a given physical-layer configuration and propagation environment. Let $\gamma_{\text{eff},t}$ denote the effective SINR associated with the t -th packet transmission, as produced by a PHY-layer simulator using detailed channel realizations, receiver processing, and decoding logic. Rather than being an independent random variable, $\{\gamma_{\text{eff},t}\}_{t \geq 1}$ exhibits strong temporal dependence due to channel time correlation, interference persistence, and receiver behavior.

We model the PHY simulator as a *teacher* that implicitly defines a conditional stochastic process:

$$\gamma_{\text{eff},t} \sim p_T(\gamma_{\text{eff},t} \mid \gamma_{\text{eff},1:t-1}, \mathcal{C}_t), \quad (1)$$

where:

- $p_T(\cdot)$ denotes the (unknown) transition law induced by the PHY simulator (teacher),
- $\gamma_{\text{eff},1:t-1}$ represents the history of effective SINR values,
- $\mathcal{C}_t$ denotes the system configuration at time t .

The configuration $\mathcal{C}_t$ may include both static parameters (e.g., channel model, antenna configuration) and dynamic parameters (e.g., modulation and coding scheme, number of

spatial streams, average SNR) that can evolve over time due to rate adaptation and link adaptation mechanisms.

The simulator-generated process in (1) is generally intractable to evaluate or sample directly without executing full PHY simulation at every time step, resulting in prohibitive computational cost for large-scale or long-horizon system-level studies.

B. Stochastic Process Knowledge Distillation

Our goal is to construct a computationally efficient *student* model that approximates the stochastic behavior of the PHY simulator while preserving essential temporal and statistical characteristics of effective SINR.

Specifically, we seek to learn a conditional stochastic process

$$\gamma_{\text{eff},t} \sim q_{\theta}(\gamma_{\text{eff},t} \mid \gamma_{\text{eff},1:t-1}, \mathcal{C}_t), \quad (2)$$

parameterized by learnable parameters θ , such that the induced process distribution closely matches that of the teacher in (1).

This formulation naturally leads to a *stochastic process knowledge distillation* problem, where the teacher is a high-fidelity PHY simulator and the student is a tractable surrogate process model. Unlike conventional knowledge distillation, which focuses on matching instantaneous outputs or marginal distributions, the objective here is to distill the *entire conditional temporal dynamics* of effective SINR.

C. Training Objective

Let $\{\gamma_{\text{eff},t}^{(n)}, \mathcal{C}_t^{(n)}\}_{t=1}^{T_n}$ denote sequences generated by the PHY simulator under possibly time-varying configurations. Here, the superscript (n) indexes independent training sequences generated by the PHY simulator, while t denotes the packet index within each sequence. Each sequence corresponds to an independent simulator run under a particular (possibly time-varying) configuration trajectory. The student model is trained to minimize the discrepancy between the teacher and student process distributions by minimizing the conditional negative log-likelihood (NLL) of simulator-generated sequences:

$$\mathcal{L}(\theta) = - \sum_n \sum_{t=1}^{T_n} \log q_{\theta}(\gamma_{\text{eff},t}^{(n)} \mid \gamma_{\text{eff},1:t-1}^{(n)}, \mathcal{C}_t^{(n)}). \quad (3)$$

Equivalently, this objective minimizes the Kullback–Leibler divergence between the teacher-induced process distribution and the student model:

$$\min_{\theta} \text{KL}(p_T(\gamma_{\text{eff},1:T} \mid \mathcal{C}_{1:T}) \parallel q_{\theta}(\gamma_{\text{eff},1:T} \mid \mathcal{C}_{1:T})), \quad (4)$$

where the dependence on the full sequence highlights the process-level nature of the distillation.

D. Comparison with Existing PHY Abstraction Paradigms

To clarify the scope and contribution of this work, we compare PHY knowledge distillation with two existing PHY abstraction paradigms: traditional link-to-system mapping-based abstraction and stochastic abstraction based on stochastic process models (e.g., EESM-log-AR). While all three aim to

enable efficient system-level simulation, they differ fundamentally in how effective SINR is obtained, what is calibrated or learned, and where computational complexity resides.

a) Traditional Link-to-System PHY Abstraction.: In traditional PHY abstraction, effective SINR is obtained through a procedural mapping applied to post-equalization SINRs generated by a high-fidelity PHY simulator. Given a full PHY configuration $\mathcal{C}$, the simulator produces post-equalization SINRs across subcarriers and spatial streams,

$$\Gamma_t = s(\mathcal{C}), \quad (5)$$

where $s(\cdot)$ denotes the PHY simulation chain including channel realization, MIMO equalization, and receiver processing. An effective SINR is then computed as

$$\gamma_{\text{eff},t} = \mathcal{A}(\Gamma_t; \boldsymbol{\eta}_{\mathcal{C}'}), \quad (6)$$

where $\mathcal{A}(\cdot)$ is a fixed link-to-system mapping (e.g., EESM), and $\boldsymbol{\eta}_{\mathcal{C}'}$ are calibration parameters indexed by a reduced configuration set $\mathcal{C}' \subset \mathcal{C}$, typically determined by the modulation and coding scheme. As a result, the offline calibration burden is relatively low. However, at inference time, effective SINR generation still requires executing the full PHY simulator $s(\mathcal{C})$, leading to high computational cost and limited scalability. Moreover, temporal correlation is not explicitly modeled but is instead inherited implicitly from the simulator.

b) Stochastic PHY Abstraction.: Stochastic abstraction methods such as EESM-log-AR replace repeated PHY simulation with an explicit stochastic process model for effective SINR. For each configuration $\mathcal{C}$, effective SINR samples generated by the simulator,

$$\{\gamma_{\text{eff},t}\}_{t=1}^T \sim s(\mathcal{C}), \quad (7)$$

are used to fit a configuration-specific stochastic process, yielding a parameter set $\Phi_{\mathcal{C}}$ that characterizes temporal dynamics and residual variability. At inference time, effective SINR is generated according to

$$\gamma_{\text{eff},t} \sim q_{\Phi_{\mathcal{C}}}(\gamma_{\text{eff},t} \mid \gamma_{\text{eff},1:t-1}), \quad (8)$$

where $\Phi_{\mathcal{C}}$ is retrieved from a lookup table indexed by $\mathcal{C}$. This approach enables fast online simulation and explicitly models temporal correlation. However, the process parameters are calibrated independently for each configuration and must be stored explicitly. Generalizing to new or adaptive configurations—such as changes in modulation, spatial streams, antenna configuration, bandwidth, or channel model—requires additional and independent offline fitting campaigns or ad hoc interpolation across stored parameter tables, which limits scalability and automation.

c) PHY Knowledge Distillation.: PHY knowledge distillation departs fundamentally from both paradigms above. Instead of calibrating abstraction parameters for each configuration or storing configuration-indexed stochastic process models, the proposed approach learns a shared, configuration-conditioned stochastic process, as formalized in (2). The learnable parameters θ are global and independent of the configuration; the configuration enters the model only as a conditioning variable, rather than as an index to stored parameters. By

training jointly over a diverse set of configurations, the student model learns how the effective-SINR stochastic process varies with system conditions and can automatically infer appropriate process behavior as the configuration changes. Consequently, effective SINR samples can be generated directly without invoking a PHY simulator, without maintaining configuration-specific parameter tables, and without separate calibration for each operating point. This shift from calibration-and-storage to inference-and-generation enables efficient simulation under time-varying configurations $\mathcal{C}_t$ and provides a scalable foundation for modeling adaptive wireless systems, including systems with resource scheduling and rate adaptation.

E. Design Requirements

The student stochastic process model must satisfy several practical requirements in order to serve as a drop-in replacement for PHY simulation in system-level studies:

- *Temporal fidelity:* accurately reproduce the temporal correlation structure and error burst behavior observed in simulator-generated effective SINR sequences;
- *Configuration adaptivity:* respond smoothly and consistently to time-varying PHY configurations, such as modulation, coding, and spatial stream adaptation;
- *Computational efficiency:* enable fast sampling and support time skipping without requiring step-by-step PHY simulation;
- *Calibration scalability:* avoid reliance on configuration-specific calibration and parameter storage, thereby reducing offline calibration effort and enabling scalable modeling across large configuration spaces.

In the following section, we introduce a structured student model that satisfies these requirements by combining an interpretable autoregressive process with learned, configuration-dependent parameter generation.

III. INSTANTIATING THE STUDENT STOCHASTIC PROCESS

In this section, we specify a concrete instantiation of the abstract student stochastic process introduced in Section II. Our objective is to construct a tractable, interpretable, and simulator-friendly approximation to

$$\gamma_{\text{eff},t} \sim q_{\theta}(\gamma_{\text{eff},t} \mid \gamma_{\text{eff},1:t-1}, \mathcal{C}_t), \quad (9)$$

where $\mathcal{C}_t$ denotes a possibly time-varying PHY configuration.

We proceed top-down. First, we impose a structured stochastic process model on the student. Second, we describe the overall architecture that generates the parameters of this process. Finally, we detail the key components that ensure stability, expressiveness, and generalization.

A. Log-Domain Autoregressive Student Process

We define the log-effective-SINR process

$$X_t \triangleq \log \gamma_{\text{eff},t}. \quad (10)$$

Working in the log domain converts multiplicative fading and interference effects in linear SINR into additive dynamics, enabling compact time-series modeling.

We instantiate the student process as a conditional, time-varying autoregressive process of order p :

$$X_t = c_t + \sum_{i=1}^p \phi_{i,t} X_{t-i} + \epsilon_t, \quad (11)$$

where the parameters $\{c_t, \phi_{i,t}\}$ and the innovation distribution of ϵ_t depend on the configuration $\mathcal{C}_t$ through learnable mappings.

Conditioned on the history $\mathbf{X}_{t-1:t-p}$, the process decomposes as

$$X_t = \mu_t + \epsilon_t, \quad \mu_t = c_t + \sum_{i=1}^p \phi_{i,t} X_{t-i}, \quad (12)$$

where μ_t captures the predictable temporal structure of log-effective-SINR and ϵ_t models residual randomness.

B. Overview of the Student Model Architecture

The student model consists of two interacting components:

- a *stochastic process model*, given by the log-AR(p) recursion in (11);
- a *neural parameter generator* that maps configuration variables $\mathcal{C}_t$ to the parameters governing this process.

Formally, the neural generator is a function

$$g_{\theta} : \mathcal{C}_t \longrightarrow (m_t, \kappa_{1:p,t}, \psi_t), \quad (13)$$

where:

- $m_t \triangleq \mathbb{E}[X_t | \mathcal{C}_t]$ is the conditional mean,
- $\kappa_{1:p,t}$ are partial autocorrelation coefficients that define the AR dynamics,
- ψ_t parameterizes the innovation distribution.

The autoregressive coefficients $\phi_{1:p,t}$ and offset c_t are then obtained deterministically from $(m_t, \kappa_{1:p,t})$, while ψ_t governs the distribution of ϵ_t . This separation allows the neural network to learn how stochastic process parameters vary with configuration, while preserving an explicit and interpretable process structure.

The student model adopts a modular architecture that separates a fixed, interpretable stochastic process from a learnable neural parameter generator. Specifically, the log-domain AR(p) recursion in (7) serves as a deterministic simulator that governs temporal evolution, while a neural network infers the parameters of this process from the PHY configuration. The neural parameter generator maps the configuration $\mathcal{C}_t$ to a compact set of stochastic process parameters, including the conditional mean, stable correlation coefficients, and innovation distribution parameters.

The configuration $\mathcal{C}_t$ is decomposed into static and dynamic components, $\mathcal{C}_t = (\mathcal{C}_{\text{static}}, \mathcal{C}_{\text{dyn},t})$. Static components $\mathcal{C}_{\text{static}} = \{\text{CH}, N_t, N_r, \text{BW}\}$ describe time-invariant structural properties of the link, while dynamic components $\mathcal{C}_{\text{dyn},t} = \{\text{SNR}_t, \text{MCS}_t, N_{\text{ss},t}\}$ capture time-varying link adaptation and operating conditions. Among these variables, only discrete mode-selection quantities—namely the channel model identifier CH, the modulation and coding scheme MCS_t , and the number of spatial streams $N_{\text{ss},t}$ —are represented using learned embeddings. These variables correspond

to distinct PHY operating regimes and may induce abrupt changes in decoding behavior and error statistics. In contrast, continuous structural parameters such as N_t , N_r , BW, and SNR_t are provided as numeric inputs, as their effects on effective SINR statistics are smooth, monotonic, and physically interpretable.

The parameter set θ comprises all learnable components of the student model. Specifically, θ includes the weights and biases of the conditioning encoder $g^{(\text{cond})}(\cdot)$, the parameter heads $g^{(\text{mean})}(\cdot)$, $g^{(\text{pacf})}(\cdot)$, and $g^{(\text{var})}(\cdot)$, as well as all embedding tables associated with discrete configuration variables such as the channel model (CH), modulation and coding scheme (MCS), and number of spatial streams (N_{ss}). All remaining components of the student stochastic process, including the autoregressive recursion, the Levinson–Durbin mapping, the mean-consistent offset computation, and the base noise distribution, are deterministic and contain no learnable parameters.

C. Neural Parameter Generator

The generator g_{θ} is implemented as a neural network composed of a conditioning encoder and multiple parameter heads. We first construct an intermediate representation $\mathbf{h}_t$ that summarizes the configuration:

$$\mathbf{h}_t = g^{(\text{cond})}(\mathcal{C}_t), \quad (14)$$

which is then mapped to process parameters via

$$(m_t, \kappa_{1:p,t}, \psi_t) = g^{(\text{head})}(\mathbf{h}_t). \quad (15)$$

The design of $\mathbf{h}_t$ and the parameter heads is described next.

1) *Compositional Conditioning Encoder*: Static configuration variables are encoded as

$$\mathbf{h}_{\text{static}} = g^{(\text{static})}(\mathcal{C}_{\text{static}}), \quad (16)$$

where $\mathbf{e}_{\text{ch}}$ is a learned embedding of the channel model.

The effect of average SNR is modeled separately using a smooth mapping

$$\mathbf{h}_{\text{snr},t} = g^{(\text{snr})}(\overline{\text{SNR}}_t). \quad (17)$$

These components define a base representation

$$\mathbf{h}_{\text{base},t} = \mathbf{h}_{\text{static}} + \mathbf{h}_{\text{snr},t}, \quad (18)$$

which captures channel physics and slowly varying link quality.

Discrete rate-adaptation variables modulate this base representation via Feature-wise Linear Modulation (FiLM):

$$\begin{aligned} \alpha_t &= g^{(\alpha)}(\mathbf{e}_{\text{mcs}}, \mathbf{e}_{\text{ss}}), \\ \beta_t &= g^{(\beta)}(\mathbf{e}_{\text{mcs}}, \mathbf{e}_{\text{ss}}), \\ \mathbf{h}_t &= \alpha_t \odot \mathbf{h}_{\text{base},t} + \beta_t, \end{aligned} \quad (19)$$

where $\mathbf{e}_{\text{mcs}}$ and $\mathbf{e}_{\text{ss}}$ are learned embeddings of the modulation and coding scheme and the number of spatial streams. This compositional structure enforces that rate adaptation modulates, rather than redefines, the underlying stochastic dynamics.

2) *Parameter Heads*: Given the conditioning representation $\mathbf{h}_t$, the parameter generator produces the parameters of the stochastic process via a set of specialized output heads. Each head is designed to reflect the semantics and constraints of the parameter it predicts.

a) *Mean head*.: The conditional mean of the log-effective-SINR process is obtained as

$$m_t = g^{(\text{mean})}(\mathbf{h}_t), \quad (20)$$

where $g^{(\text{mean})}(\cdot)$ is a lightweight multilayer perceptron (MLP) with a scalar output. This head models the slowly varying average behavior of the link and primarily captures the dependence on average SNR and large-scale configuration variables.

For a locally stationary AR(p) process, the mean satisfies

$$m_t = c_t + \sum_{i=1}^p \phi_{i,t} m_t, \quad (21)$$

which yields the offset

$$c_t = \left(1 - \sum_{i=1}^p \phi_{i,t}\right) m_t. \quad (22)$$

This construction ensures consistency between the mean behavior and correlation structure of the process.

b) *Autoregressive dynamics head*.: Temporal correlation is parameterized via partial autocorrelation coefficients. We define

$$\mathbf{u}_{1:p,t} = g^{(\text{pacf})}(\mathbf{h}_t), \quad (23)$$

where $g^{(\text{pacf})}(\cdot)$ is an MLP with p outputs. These unconstrained values are mapped to valid partial autocorrelations by

$$\kappa_{i,t} = (1 - \delta) \tanh(u_{i,t}), \quad i = 1, \dots, p, \quad (24)$$

ensuring $\kappa_{i,t} \in (-1, 1)$. The autoregressive coefficients $\phi_{1:p,t}$ are then obtained deterministically from $\kappa_{1:p,t}$ using the Levinson–Durbin recursion, which guarantees stability of the resulting AR(p) process.

c) *Innovation distribution head*.: Residual variability in the log-effective-SINR process is modeled using a configuration-dependent Gaussian innovation. Specifically, we introduce a dedicated innovation head that maps the conditioning representation $\mathbf{h}_t$ to the scale parameter of a zero-mean Gaussian distribution:

$$\sigma_t = g^{(\text{var})}(\mathbf{h}_t), \quad (25)$$

where $g^{(\text{var})}(\cdot)$ outputs an unconstrained real value that is transformed into a strictly positive standard deviation using a softplus mapping.

Conditioned on $\mathbf{h}_t$, the innovation process is defined as

$$\epsilon_t = \sigma_t z_t, \quad z_t \sim \mathcal{N}(0, 1). \quad (26)$$

This formulation allows the magnitude of residual uncertainty to adapt smoothly to configuration changes, while preserving a simple and interpretable innovation structure.

IV. MODEL TRAINING

This section derives the training objective for the proposed student stochastic process and presents an implementation-oriented training algorithm. We assume access to sequences generated by the PHY simulator (teacher), consisting of effective SINR samples $\{\gamma_{\text{eff},t}\}$ and the associated configuration trajectory $\{\mathcal{C}_t\}$.

A. Training Objective

Training the student model amounts to minimizing the discrepancy between the teacher-induced effective-SINR process and the parameterized student process described in Section III. Given simulator-generated sequences $\{X_t^{(n)}, \mathcal{C}_t^{(n)}\}_{t=1}^{T_n}$ and an autoregressive memory of order p , the knowledge-distillation objective in (3) reduces to minimizing the conditional negative log-likelihood

$$\mathcal{L}(\theta) = - \sum_n \sum_{t=p+1}^{T_n} \log q_\theta \left(X_t^{(n)} \mid \mathbf{X}_{t-p:t-1}^{(n)}, \mathcal{C}_t^{(n)} \right), \quad (27)$$

where θ denotes the global learnable parameters of the student model.

Under the log-domain AR(p) formulation in Section III, the student model specifies the conditional distribution of the next log-effective-SINR sample as

$$q_\theta(X_t \mid \mathbf{X}_{t-p:t-1}, \mathcal{C}_t) = \mathcal{N}(X_t; \mu_t, \sigma_t^2), \quad (28)$$

where the conditional mean μ_t is determined by the autoregressive dynamics and depends on the history $\mathbf{X}_{t-p:t-1}$ as well as the configuration-dependent parameters predicted from $\mathcal{C}_t$, while the standard deviation σ_t is produced by the innovation head described in the previous subsection.

Recalling the innovation definition $\epsilon_t = X_t - \mu_t$, and using the Gaussian innovation model in (26), the conditional log-likelihood admits the closed-form expression

$$\log q_\theta(X_t \mid \mathbf{X}_{t-p:t-1}, \mathcal{C}_t) = -\frac{1}{2} \left(\frac{\epsilon_t}{\sigma_t} \right)^2 - \log \sigma_t - \frac{1}{2} \log(2\pi), \quad (29)$$

which is used directly as the training objective. Substituting (29) into (27), the training objective becomes

$$\mathcal{L}(\theta) = \sum_n \sum_{t=p+1}^{T_n} \left[\frac{1}{2} \left(\frac{\epsilon_t^{(n)}}{\sigma_t^{(n)}} \right)^2 + \log \sigma_t^{(n)} \right] + \text{const}, \quad (30)$$

where the constant term does not depend on the model parameters.

This objective jointly trains all components of the student model. The mean and autoregressive heads learn to capture predictable temporal structure in the log-effective-SINR sequence, while the innovation head adapts the scale of residual uncertainty in a configuration-dependent manner. Empirically, this Gaussian formulation yields well-calibrated standardized innovations, as verified by probability integral transform diagnostics, and provides sufficient fidelity for the PHY scenarios considered in this work.

Algorithm 1 Training the Distilled Student Process via Sequence Maximum Likelihood

```

1: Input: Dataset  $\{(\gamma_{\text{eff},1:T}^{(n)}, \mathcal{C}_{1:T}^{(n)})\}_{n=1}^N$ , AR order  $p$ , parameter
   generators  $g_{\theta}^{(\text{cond})}(\cdot)$  and  $g_{\theta}^{(\text{head})}(\cdot)$ , optimizer (e.g., Adam).
2: Output: Learned parameters  $\theta$ .
3: for each minibatch of sequences  $\{(\gamma_{\text{eff},1:T}^{(n)}, \mathcal{C}_{1:T}^{(n)})\}$  do
4:   Convert targets to log domain:  $X_t^{(n)} \leftarrow \log \gamma_{\text{eff},t}^{(n)}$  for all  $t$ .
5:   Initialize minibatch loss  $\mathcal{J} \leftarrow 0$ .
6:   for each sequence  $n$  in minibatch do
7:     for  $t = p + 1$  to  $T_n$  do
8:       (1) Encode configuration:  $\mathbf{h}_t^{(n)} \leftarrow g_{\theta}^{(\text{cond})}(\mathcal{C}_t^{(n)})$ .
9:       (2) Predict process parameters:  $(m_t^{(n)}, \kappa_{1:p,t}^{(n)}, \sigma_t^{(n)}) \leftarrow$ 
           $g_{\theta}^{(\text{head})}(\mathbf{h}_t^{(n)})$ .
10:      (3) Stable AR construction:  $\phi_{1:p,t}^{(n)} \leftarrow$ 
          LevinsonDurbin( $\kappa_{1:p,t}^{(n)}$ ),
11:       $c_t^{(n)} \leftarrow (1 - \sum_{i=1}^p \phi_{i,t}^{(n)}) m_t^{(n)}$ .
12:      (4) Teacher-forced AR mean:
13:       $\mu_t^{(n)} \leftarrow c_t^{(n)} + \sum_{i=1}^p \phi_{i,t}^{(n)} X_{t-i}^{(n)}$ .
14:      (5) Innovation from residual:  $\epsilon_t^{(n)} \leftarrow X_t^{(n)} - \mu_t^{(n)}$ .
15:      (6) Accumulate Gaussian negative log-likelihood:
16:       $\mathcal{J} \leftarrow \mathcal{J} + \frac{1}{2} \left( \frac{\epsilon_t^{(n)}}{\sigma_t^{(n)}} \right)^2 + \log \sigma_t^{(n)}$ .
17:     end for
18:   end for
19:   Update parameters:
20:    $\theta \leftarrow \text{OptimizerStep}(\theta, \nabla_{\theta} \mathcal{J})$ .
21: end for

```

B. Training Data

Training sequences are generated using a high-fidelity PHY simulator by recording effective-SINR samples $\{\gamma_{\text{eff},t}, \mathcal{C}_t\}$ at a fixed packet sampling period T_s . To balance temporal correlation and sample diversity, we set

$$T_s = \frac{1}{4} T_c, \quad (31)$$

where T_c denotes the channel coherence time.

If $T_s \ll T_c$, successive log-effective-SINR samples are nearly identical, resulting in highly redundant training sequences with limited information for learning autoregressive dynamics. If $T_s \approx T_c$, consecutive samples become weakly correlated and temporal correlation information is lost. The chosen sampling period provides informative temporal transitions while preserving channel-induced correlation, enabling stable learning of the AR structure and innovation distribution.

C. Training Algorithm

Algorithm 1 summarizes the training procedure. Unlike inference (Algorithm 2), training uses *teacher-forcing*: the AR recursion uses the ground-truth history $\mathbf{X}_{t-p:t-1}$ from the teacher sequence rather than sampled student states.

V. MODEL INFERENCE

At inference time, the learned student model replaces per-packet PHY simulation by directly generating effective SINR samples and packet error events. The procedure is fully compatible with standard PHY abstraction pipelines based on AWGN lookup tables.

A. Initialization

The AR(p) recursion in Algorithm 2 requires an initial state history $\{X_0, X_{-1}, \dots, X_{-p+1}\}$, which is not available at the start of a simulation run. We adopt a simple and robust initialization strategy based on conditional mean initialization with burn-in.

At the start of the simulation, the state history is initialized using the conditional mean predicted from the initial configuration:

$$X_0 = X_{-1} = \dots = X_{-p+1} = m_1. \quad (32)$$

The student process is then evolved forward for T_{burn} packets, and the generated samples are discarded. This burn-in phase allows the autoregressive state to forget the artificial initial condition and transition to configuration-consistent stochastic dynamics before normal simulation proceeds.

B. Time-Skipping via Configuration-Driven Parameter Caching

System-level simulators operate in an event-driven manner, where PHY configurations are updated infrequently (e.g., due to scheduling or rate adaptation), while packet transmissions occur at a finer time scale. An effective PHY abstraction must therefore support efficient time skipping between configuration updates.

The proposed student model decouples configuration-dependent parameter inference from temporal evolution. When the PHY configuration remains unchanged over a window of packets, $\mathcal{C}_t = \mathcal{C}_{t+1} = \dots = \mathcal{C}_{t+\Delta}$, the parameters of the conditional log-domain AR process inferred from the configuration are constant over the entire interval. In this case, the neural parameter generator is evaluated only once, and the simulator advances the effective-SINR process using cached AR and innovation parameters.

Between configuration updates, effective SINR samples are generated by propagating the fixed AR recursion with independently sampled innovations, exactly simulating the learned student process without repeated neural inference. As a result, time skipping incurs only $O(p)$ scalar operations per packet and avoids channel realization generation, post-equalization SINR computation, and per-packet network evaluation, making the proposed approach naturally compatible with large time skips and scalable event-driven system-level simulation.

C. Inference Algorithm

Algorithm 2 summarizes the inference procedure executed at each packet time index t . The above procedure generates a temporally correlated effective SINR sequence whose statistics match those of the PHY simulator, while preserving compatibility with conventional AWGN-based packet error modeling.

VI. EXPERIMENTAL RESULTS

Empirically, we find that Gaussian innovations with configuration-dependent variance are sufficient to capture the residual statistics of the underlying PHY simulator across a wide range of operating points. In particular, probability

Algorithm 2 Inference: Generate Effective SINR and Packet Errors Using the Distilled Student Process

- 1: **Input:** Configuration trajectory $\{\mathcal{C}_t\}_{t=1}^T$ (may vary with rate adaptation), AR order p , burn-in length T_{burn} , trained student networks $g_{\theta}^{(\text{cond})}(\cdot)$ and $g_{\theta}^{(\text{head})}(\cdot)$, monotone spline flow transform $f_{\psi(\cdot)}(\cdot)$, AWGN PER lookup function $\text{PER}_{\text{AWGN}}(\cdot; \text{MCS})$.
 - 2: **Output:** Effective SINR samples $\{\gamma_{\text{eff},t}\}_{t=1}^T$, packet error events $\{E_t\}_{t=1}^T$.
 - 3: **Initialization:** Construct initial log-state history $\{X_0, X_{-1}, \dots, X_{-p+1}\}$.
 - 4: Compute $h_1 \leftarrow g_{\theta}^{(\text{cond})}(\mathcal{C}_1)$ and $(m_1, \kappa_{1:p,1}, \psi_1) \leftarrow g_{\theta}^{(\text{head})}(h_1)$.
 - 5: Initialize history using predicted mean:
 - 6: $X_0 = X_{-1} = \dots = X_{-p+1} = m_1$.
 - 7: **Burn-in:** For $t = 1$ to T_{burn} , run Steps (1)–(8) below and discard samples; keep the latest p states.
 - 8: **for** $t = 1$ to T **do**
 - 9: **(1) Encode current configuration:** $h_t \leftarrow g_{\theta}^{(\text{cond})}(\mathcal{C}_t)$.
 - 10: **(2) Predict process parameters:** $(m_t, \kappa_{1:p,t}, \psi_t) \leftarrow g_{\theta}^{(\text{head})}(h_t)$.
 - 11: **(3) Construct AR coefficients:** $\phi_{1:p,t} \leftarrow \text{LevinsonDurbin}(\kappa_{1:p,t})$, $c_t \leftarrow \left(1 - \sum_{i=1}^p \phi_{i,t}\right)m_t$.
 - 12: **(4) Sample innovation:** Draw $z_t \sim \mathcal{N}(0, 1)$ and set $\epsilon_t \leftarrow f_{\psi_t}(z_t)$.
 - 13: **(5) Propagate log-AR(p):** $\mu_t \leftarrow c_t + \sum_{i=1}^p \phi_{i,t}X_{t-i}$; $X_t \leftarrow \mu_t + \epsilon_t$.
 - 14: **(6) Convert to effective SINR:** $\gamma_{\text{eff},t} \leftarrow \exp(X_t)$.
 - 15: **(7) Map effective SINR to PER:** $p_{e,t} \leftarrow \text{PER}_{\text{AWGN}}(\gamma_{\text{eff},t}; \text{MCS}_t)$.
 - 16: **(8) Sample packet error:** $E_t \sim \text{Bernoulli}(p_{e,t})$.
 - 17: **end for**
-

REFERENCES

integral transform (PIT) diagnostics confirm that the standardized innovations are well-calibrated and approximately i.i.d. Gaussian, indicating that more complex innovation models provide limited additional benefit in the scenarios considered.

VII. CONCLUSION

This paper introduced PHY Knowledge Distillation (PKD), a stochastic abstraction framework that replaces configuration-specific PHY calibration with inference-driven parameter generation. By modeling effective SINR as a configuration-conditioned stochastic process, PKD shifts PHY abstraction from a calibration-and-storage paradigm to an inference-and-generation paradigm, eliminating the need for per-configuration parameter tables.

The proposed approach retains an explicit and interpretable autoregressive structure while learning how its parameters vary smoothly with system configuration. Temporal correlation is captured through a stable log-domain AR process, residual innovation effects are modeled via conditional monotone normalizing flows, and compositional conditioning enables robust adaptation to time-varying link configurations. This structure allows exact and efficient time skipping through parameter caching, making the framework well suited for large-scale, event-driven system-level simulation. PKD demonstrates how stochastic process modeling and learning-based parameterization can be combined to achieve scalability, interpretability, and computational efficiency in PHY abstraction.